

DOCUMENTATION TECHNIQUE

V i t e & G o u r m a n d

Projet ECF - Application web et web mobile de traiteur événementiel

Titre professionnel Graduate Développeur Web et Web Mobile

Sofiène Guernane
Septembre 2026

Sommaire

1. Présentation générale du projet
2. Architecture applicative et organisation MVC
3. Environnement technique
4. Configuration de l'environnement de travail
5. Structure de la base de données SQL/NoSQL et MCD
6. Sécurité et mesures de protection
7. Flux fonctionnels clés
8. Diagramme de cas d'utilisation
9. Diagramme de séquence
10. Intégrations externes
11. Performances et optimisation
12. Procédure de déploiement
13. Maintenance et bonnes pratiques

1. Présentation générale du projet

Vite & Gourmand est une application web et web mobile de traiteur événementiel conçue dans le cadre d'un projet ECF. Elle permet aux clients de commander des menus en ligne pour une date et un lieu de prestation donnés, tout en offrant à une équipe staff (administrateurs et employés) des outils complets de gestion des menus, des plats, des commandes, des avis et des comptes utilisateurs.

Le point d'entrée unique (`public/index.php?page=...`) s'appuie sur un routeur structuré en matchet un contrôle d'accès strict défini dans `app/Config/pages.php`.

1.1 Réflexions technologiques initiales

Le sujet ECF n'imposait aucune technologie hormis l'usage d'une base de données relationnelle (SQL) et d'une base de données non relationnelle (NoSQL).

Les choix retenus reposent donc sur des critères pragmatiques adaptés au contexte d'un projet solo avec délai contraint :

- **PHP 8.2**: permet de démontrer la maîtrise des mécanismes MVC, PDO, sessions et sécurité qui correspond à l'objectif pédagogique du référentiel DWWM.
- **MariaDB pour le relationnel** : adapté à des données fortement structurées et relationnelles (menus, plats, commandes, utilisateurs) avec de nombreuses contraintes d'intégrité (clés étrangères, unicité).
- **MongoDB pour le NoSQL** : utilisé pour les statistiques, données moins structurées et lues en agrégation (chiffre d'affaires par menu/période) comme demandé dans l'ECF.
- **Composer** : accès simple à des librairies tierces ciblées (Brevo, Guzzle, MongoDB), sans alourdir le projet. L'autoloading applicatif (`App\*`) est assuré par un autoloader custom (`App\Autoloader`), pas par `Composer/PSR-4` -voir section Configuration Bootstrap 5 + JS vanilla : l'app reste rendue côté serveur avec des interactions AJAX ciblées (filtres, recalcul de prix, ajout de plats).
- **OpenRouteService + API Adresse (data.gouv.fr)** : solutions gratuites/open-data pour le calcul de distance routière et l'autocomplétion d'adresse française, afin d'éviter une dépendance payante (type Google Maps) pour un projet pédagogique.
- **Brevo pour les emails transactionnels** : offre un plan gratuit suffisant pour le volume ECF et un SDK PHP officiel, alternative à `mail()` natif peu fiable en production.

1.2 Profils d'utilisateurs et espaces fonctionnels

Rôle	Constante	Accès principaux
Visiteur	ROLE_VISITEUR = 0	Accès = Accueil, catalogue, contact, CGV, mentions légales
Client	ROLE_USER = 3	Accès Visiteur + Parcours de commande, dashboard, avis.
Employé	ROLE_EMPLOYE = 2	Accès au dashboard opérationnel partagé pour la gestion des menus, des commandes, des plats et un premier niveau de modération des avis.
Administrateur	ROLE_ADMIN = 1	Accès incluant l'intégralité des fonctionnalités opérationnelles de l'employé, avec en plus la gestion des ressources humaines, les statistiques, ainsi qu'un droit de correction sur la modération des avis.

1.3 Ventilation détaillée des espaces fonctionnels :

Espace public / visiteur : Accueil, catalogue des menus avec moteur de recherche et filtres, page de contact, pages légales (Mentions légales, CGV, RGPD), formulaires d'authentification et de réinitialisation de mot de passe.

Espace client (ROLE_USER) : Parcours de commande guidé en 6 étapes (order-menu), tableau de bord et gestion du profil (dashboard-user), outils de modification et de recalcul dynamique (edit-order, update-order, recalculer-prix, cancel-edit-order, erase-order), et espace de dépôt des avis (review, store-review).

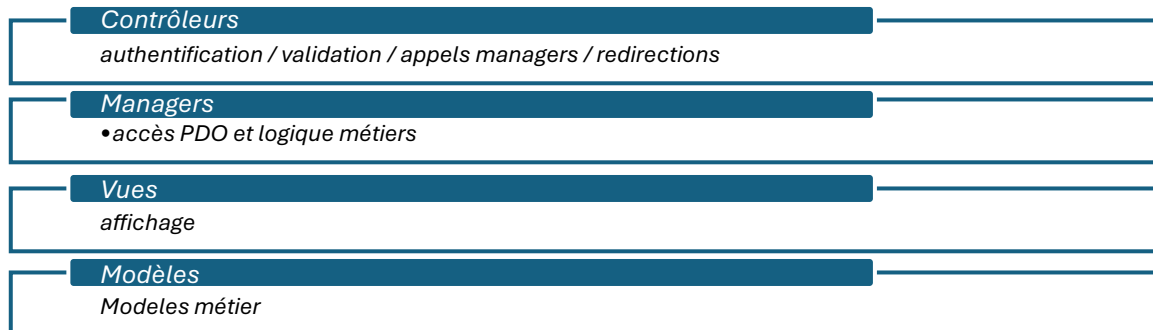
Espace employé (ROLE_EMPLOYE) : Accès au tableau de bord opérationnel dédié (dashboard-employee).

Espace administrateur (ROLE_ADMIN) : Pilotage global (dashboard-admin), statistiques analytiques (stats-admin), et gestion des ressources humaines et des sanctions (rh-admin : création, suppression, bascule de statut, bannissement).

Espace staff commun (Admin + Employé) : Espace de travail mutualisé regroupant la gestion des menus et des plats (CRUD), le suivi et la mise à jour des statuts de commande (order-management, update-order-status), ainsi que la modération des avis clients (review-management).

2. Architecture applicative et organisation MVC

Le projet applique une structure Modèle-Vue-Contrôleur (MVC) rigoureuse en PHP natif, répartie à travers une arborescence de dossiers claire et cloisonnée :



public/index.php	Front controller : session sécurisée, CSRF, routeur, contrôle d'accès
app/Config/	Constants.php, pages.php, env.php, Database.php
app/Controllers/	Orchestration HTTP uniquement : AdminController/ AuthController/ EmployeeController/ StaffCommon/ UserController
app/Managers/	SQL (requêtes préparées) : MenuManager, OrderManager, PlatManager, LieuManager, UserManager, UserAdminManager, ReviewManager, HoraireManager, StatAdminManager, MongoStatsManager
app/Models/	Entités + logique métier (Menu::calculerTotal, Order distance/frais)
app/Views/	Templates PHP (layouts header/footer + partials + vues par rôle)
app/Helpers/	SecurityManager (CSRF), Function.php (flash), MailService
public/assets/	css/js/img (+ uploads plats)

3. Environnement technique et stack logicielle

L'application repose sur une architecture orientée objet en PHP natif (sans framework lourd), structurée autour de choix technologiques robustes pour la gestion des données, de la sécurité et des services externes :

Élément	Détails
PHP	8.2 (XAMPP en local) / 8.4 (Docker et production Heroku)
Base de données relationnelle	MySQL 8.2 / MariaDB 10.6+ (test_transit_ecf), préfixe de table vg_
Base de données NoSQL	MongoDB atlas (statistiques)
Gestionnaire de dépendances	Composer (composer.json)
Autoloading	App\Autoloader (spl_autoload_register custom, chargé manuellement dans index.php)
Rôles	ADMIN=1, EMPLOYE=2, USER=3, VISITEUR=0
Constantes clés	BASE_URL, ROOT_PATH, DEPOT_GARANTIE_MATERIEL=600.00, COMPANY_LAT=44.837789, COMPANY_LON=-0.579180
Secrets	Stockés dans .env (non committé), chargés par app/Config/env.php (putenv + \$_ENV)
Frameworks/libs front	Bootstrap 5, FontAwesome, DataTables, JS vanilla
API externes	API Adresse (data.gouv.fr), OpenRouteService, Brevo

3.1 Dépendances et gestion des paquets

Les packages tiers indispensables au projet sont gérés via Composer et centralisés dans le fichier composer.json :

```
{
  "require": {
    "mongodb/mongodb": "^2.4",
    "getbrevo/brevo-php": "^5.0",
    "guzzlehttp/guzzle": "^8.1"
  }
}
```

3.2 Gestion des URL (BASE_URL vs APP_URL)

L'application distingue deux configurations d'URL pour répondre aux besoins de routage interne et de communication externe :

- **BASE_URL** est calculée dynamiquement à partir de la variable d'environnement **APP_ENV** : elle vaut `/Projet_Vite_Gourmand_Finale/public` uniquement en local XAMPP (`APP_ENV=local`), et reste vide (") pour tous les autres environnements (Docker, production Heroku), où `public/` constitue déjà la racine web. Ce mécanisme centralise la logique dans `Constants.php`, qui charge `env.php` avant de calculer `BASE_URL`, garantissant une valeur cohérente peu importe l'environnement d'exécution. Tous les chemins d'assets (`CSS/JS/images`) sont construits en relatif à partir de cette constante. Ce mécanisme a été généralisé lors de l'intégration de Docker comme troisième environnement d'exécution : la variable `APP_ENV` est désormais explicitement définie dans chaque contexte (fichier `.env` local pour XAMPP, `docker-compose.yml` pour Docker, `Config Vars` Heroku pour la production), évitant toute ambiguïté sur l'environnement réellement actif.
- **APP_URL** (variable d'environnement du `.env`) : utilisée exclusivement pour générer des liens absolus (notamment dans les e-mails transactionnels de réinitialisation de mot de passe via `ForgotPasswordController`).

4. Configuration de l'environnement de travail

Étapes pour installer et exécuter l'application en local (environnement XAMPP / MariaDB / PHP) :

4.1 Prérequis

- OS : Windows avec XAMPP (Apache & MariaDB)
- PHP : version 8.2 ou supérieure
- Composer (gestionnaire de dépendances)
- Base SQL : MariaDB (fournie avec XAMPP en local)
- Base NoSQL : MongoDB Atlas
- Extensions PHP requises : `pdo_mysql`, `mongodb`, `mbstring`, `curl`, `fileinfo`
- Clé d'API OpenRouteService (via heiGIT) pour le calcul d'itinéraire de livraison

4.2 Installation locale (6 étapes)

1. Placer le projet dans XAMPP

`C:\xampp\htdocs\Projet_Vite_Gourmand_Finale`

Si le dossier change de nom, adapter le chemin local dans la condition définissant `BASE_URL` (fichier `app/Config/Constants.php`), qui reste `/Projet_Vite_Gourmand_Finale/public` par défaut pour l'environnement local (`APP_ENV=local`).

2. Installer les dépendances PHP

Installer également la bibliothèque MongoDB dans `vendor/`.

```
composer install
```

3. Créer la base MariaDB

- Ouvrir phpMyAdmin (<http://localhost/phpmyadmin/>)
- Créer une base nommée `test_transit_ecf`
- Importer le script : `app/Database/Schema_VG.sql`
- Vérifier que les tables importées utilisent bien le préfixe `vg_`
- Importer le script de données test : `app/Database/fixture_VG.sql`

4. Créer l'utilisateur dédié

L'application se connecte via un utilisateur dédié plutôt que via le compte root :

```
CREATE USER 'vg_creator'@'localhost' IDENTIFIED BY 'votre_mot_de_passe';
GRANT ALL PRIVILEGES ON test_transit_ecf.* TO 'vg_creator'@'localhost';
FLUSH PRIVILEGES;
```

5. Configurer les variables d'environnement (.env)

Créer un fichier .env à la racine du projet : **IMPORTANT : Placer .env dans votre fichier « gitignore », Git ne doit jamais contenir de secrets committés.**

```
# Base de données relationnelle
DB_HOST=localhost
DB_NAME=test_transit_ecf
DB_USER=vg_creator
DB_PASS= votre_mot_de_passe

# Base de données NoSQL
MONGODB_URI=mongodb+srv://<votre_username>:<votre_mot_de_passe>@<cluster>.mongodb.net/vite_gourmand?appName=ProjetViteGourmand

# API externes & services
BREVO_API_KEY=votre_cle_brevo
MAIL_FROM_EMAIL=votre_email
OPENROUTESERVICE_API_KEY=votre_cle_openroute

# URL absolue de l'application, utilisée pour générer les liens dans les emails (ex. réinitialisation de mot de passe)
APP_URL=http://localhost/Projet_Vite_Gourmand_Finale/public
```

Démarrer MongoDB avant d'utiliser les statistiques administrateur.

6. Configurer les emails

L'application utilise exclusivement l'API Brevo pour l'envoi d'emails transactionnels (confirmation de commande, réinitialisation de mot de passe, création de compte staff), y compris en développement local, aucune configuration SMTP locale (MailHog, mail() natif) n'est nécessaire.

Renseigner simplement **BREVO_API_KEY** dans le fichier .env.

Sans cette clé, MailService::sendEmail() échoue silencieusement et journalise l'erreur (error_log) sans bloquer le reste de l'application.

4.3 Lancer l'application

http://localhost/Projet_Vite_Gourmand_Finale/public/index.php?page=home

Paramètre	Page
?page=home	Accueil
?page=search	Liste des menus
?page=contact	Contact
?page=login	Connexion
?page=signin	Création de compte
?page=mention	Mentions légales et CGV

4.4 Comptes de test par défaut

Rôle	Identifiant
Administrateur	admin@vite-gourmand.fr
Employé	employe@vite-gourmand.fr
Client	client@vite-gourmand.fr

4.5 Installation via Docker

En complément de l'installation locale via XAMPP, le projet dispose d'une configuration Docker complète, permettant de lancer l'application dans un environnement conteneurisé et reproductible (PHP 8.4/Apache + MariaDB).

Prérequis :

Docker Desktop installé et démarré

Fichiers fournis :

- Dockerfile : image applicative (PHP 8.4/Apache, extensions pdo, pdo_mysql, mbstring, mongodb compilée avec support SSL)
- docker-compose.yml : orchestration de deux services (app sur le port 8080, db MariaDB 10.6 sur le port 3307), avec volumes persistants
- .env.docker (non versionné) : variables d'environnement propres à Docker

Lancement :

```
docker compose --env-file .env.docker up --build
```

L'application est alors accessible sur <http://localhost:8080/index.php?page=home>.

5. Structure de la base de données

15 tables composent le schéma relationnel actif. Toutes les tables sont préfixées `vg_`.

Table	Description
vg_utilisateur vg_role	Comptes (email unique, mdp bcrypt, rôle, statut) et rôles disponibles
vg_menu vg_plat	Catalogue des menus et des plats (photo WebP en chemin relatif : assets/img/plats/xxx.webp) -aucun BLOB n'est utilisé, ce qui garde le dump SQL léger et permet de versionner les images dans Git.
vg_menu_plat	Table pivot menus ↔ plats
vg_theme vg_regime	Thèmes des menus et régimes alimentaires
vg_allergene vg_allergene_plat	Allergènes et table pivot plats ↔ allergènes
vg_commande	Commandes clients (statut, prix, dates, lieu, menu, dépôt de garantie)
vg_commande_statut_historique	Historique horodaté des changements de statut d'une commande
vg_lieu_prestation	Adresses de prestation (ville, code postal, GPS, distance à Bordeaux)
vg_avis	Avis client liés à une commande (une seule contrainte : 1 avis par commande)
vg_horaire	Horaires d'ouverture
vg_password_resets	Tokens de réinitialisation de mot de passe avec expiration / un seul token actif par email (contrainte UNIQUE)

5.1 Modèle conceptuel de données (MCD)

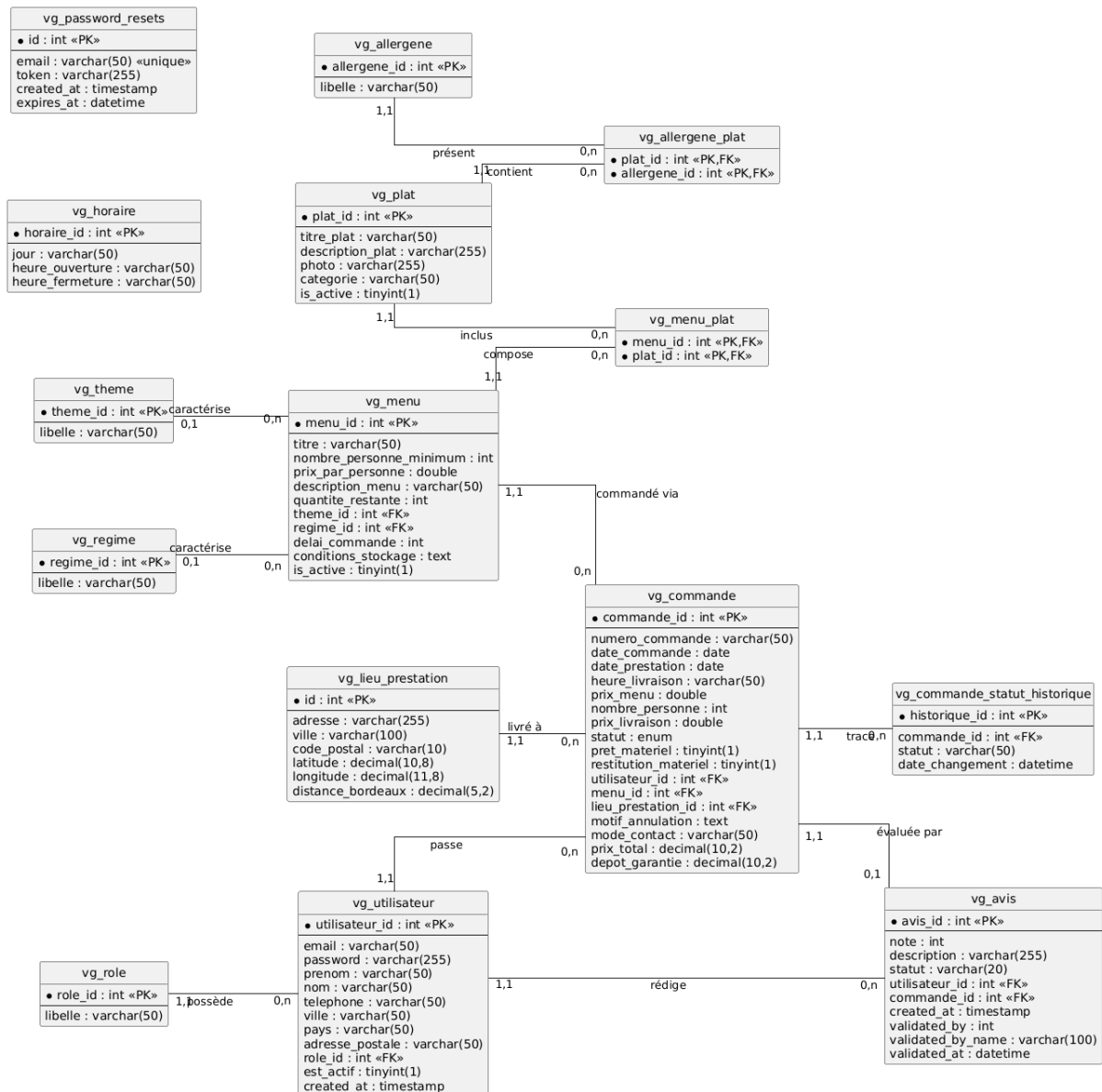


Figure 1 - MCD complet de Vite & Gourmand (15 tables, cardinalités et clés déduites du dictionnaire de données réel)

Les cardinalités sont exprimées en notation Merise (min,max) et ont été déduites manuellement de la structure réelle de la base (nullabilité des clés étrangères, contraintes d'unicité), plutôt que générées automatiquement par un outil de rétro-ingénierie qui se contenterait de détecter les clés étrangères sans analyser leurs contraintes.

Le diagramme a ensuite été mis en forme via PlantUML à partir de ces valeurs.

Points structurants :

- `vg_menu_plat` et `vg_allergene_plat` sont des tables de liaison, matérialisant les relations many-to-many entre menus/plats et plats/allergènes.
- La relation entre `vg_commande` et `vg_avis` est bornée à 0,1 côté avis, reflet de la contrainte d'unicité `une_seule_avis_par_commande` en base.
- `vg_theme` et `vg_regime` sont reliées à `vg_menu` en cardinalité 0,1 côté menu, ces deux colonnes étant nullables dans le schéma.
- `vg_horaire` et `vg_password_resets` sont des tables autonomes, sans clé étrangère.

5.2 Modélisation et intégration de la base NoSQL (MongoDB)

L'architecture hybride du projet répartit les rôles entre **MariaDB** (cœur transactionnel : utilisateurs, catalogues, stocks, intégrité des commandes) et **MongoDB** (suivi analytique et statistiques du tableau de bord admin).

5.3 Pourquoi ce choix

L'utilisation de MongoDB évite de surcharger la base relationnelle avec de lourdes agrégations en temps réel et offre deux avantages majeurs :

- **Performance et isolation** : Le dashboard admin interroge une base orientée document dédiée, préservant les performances transactionnelles des utilisateurs.
- **Historisation figée ("Snapshot")** : Chaque document stocke une copie figée des données au moment de l'achat (client, articles, prix). L'historique des ventes reste ainsi totalement cohérent même en cas de modification ou suppression ultérieure dans le catalogue SQL.

5.4 Structure du document NoSQL

À chaque validation de commande, un document analytique est généré dans la collection MongoDB:

```
{
  "_id": "6a9c48407727621f890709ee",
  "commande_id_sql": null,
  "client": {
    "nom": "DUPONT Jean",
    "email": "jean.dupont@example.com"
  },
  "articles": [
    {
      "menu_id": 1,
      "titre": "Menu Découverte",
      "quantite": 3,
      "prix_unitaire": 36,
      "sous_total": 108
    }
  ],
  "montant_details": {
    "prix_menus": 108,
    "frais_livraison": 15.00,
    "depot_garantie_caution": 600,
    "ca_reel_entreprise": 123.00,
    "montant_total_paye_par_client": 723.00
  },
  "date_creation": "2026-09-05T16:50:08.438Z"
}
```

5.5 Implémentation technique

- **Connexion** : Gérée via le package officiel mongodb/mongodb via Composer, connectée à un cluster cloud **MongoDB Atlas** dont l'URI est sécurisée et centralisée dans le fichier .env.
- **Exploitation** : Les scripts du panneau d'administration lisent directement cette collection sur Atlas pour alimenter les graphiques de suivi du chiffre d'affaires et des ventes par menu.

6. Sécurité et mesures de protection

6.1 Sécurisation technique et traitement des menaces

Le tableau suivant répertorie les principales menaces web et détaille les contre-mesures techniques concrètement implémentées dans le code de l'application :

Menace	Mesure appliquée
Injections SQL	100% des accès DB via requêtes préparées paramétrées, centralisées dans les Managers ; 0 SQL direct dans contrôleurs/vues
XSS	htmlspecialchars(..., ENT_QUOTES, 'UTF-8') systématique côté serveur sur toute donnée provenant d'une saisie utilisateur avant affichage dans les vues ; Côté client, les insertions dynamiques via innerHTML (affichage des menus en JavaScript) sont échappées via une fonction dédiée avant insertion dans le DOM
CSRF	Jeton unique par session (\$_SESSION['csrf_token']), validé via SecurityManager::validatePost() (hash_equals) sur les 22 formulaires POST de l'application.
Sessions	Options HttpOnly, SameSite=Lax, Secure (auto si HTTPS), session.use_strict_mode activé (protection contre la fixation de session)
Mots de passe	Hachage bcrypt (password_hash()) ; reset par token aléatoire bin2hex(random_bytes(32)) + expiration, suppression après usage
Réinitialisation mdp	vg_password_resets contraint email en UNIQUE (uk_email) : un seul token actif par adresse email, upsert obligatoire à chaque nouvelle demande plutôt qu'un INSERT simple
Uploads	Uploads Vérification MIME (finfo, jpg/png/webp uniquement), taille max 5 Mo, nom aléatoire bin2hex(random_bytes(16)) ; fichiers stockés hors racine web (storage/uploads/plats/), livrés via une route contrôlée (MediaController::servePlatImage) avec validation stricte du nom de fichier et repli automatique sur une image par défaut
Contrôle d'accès	Auth::check([ROLE_...]) en tête de chaque action sensible ; matrice de routage pages.php via pages existante pages publiques et pages par rôles ; un admin ne peut pas être banni
Adresses / GPS	LieuManager::getOrInsert() rejette les lieux incomplets ; adresse validée par API + regex côté serveur
Secrets	.env / .env.docker non committé, chargés via app/Config/env.php

6.2 Matrice des rôles et contrôle d'accès

La matrice ci-dessous résume le découpage des rôles, la ventilation des routes et les périmètres fonctionnels associés dans le système : »

Catégorie / Rôle	Pages / Routes associées	Description du périmètre
Publiques <i>(Visiteurs non connectés)</i>	home, login, signin, search, details-menu, filter, contact, contact-success, mention, 404, error-ban, forgot-password, reset-password	Pages de consultation, d'authentification, de recherche et d'information accessibles à tout public.
Client <i>(Utilisateur standard)</i>	dashboard-user, update-profil, order-menu, order-success, erase-order, ajax-frais-livraison, edit-order, recalculer-prix, cancel-edit-order, update-order, review, store-review, deactivate-account	Espace personnel du client, parcours de commande de A à Z, gestion des avis et du profil.
Employé	dashboard-employee	Tableau de bord opérationnel dédié à l'espace de travail de l'employé.
Staff <i>(Employé + Administrateur)</i>	menu-management, order-management, cancel-order-common, edit-order-common, update-order-common, update-order-status, edit-menu, update-menu-process, add-menu-process, create-plat-ajax, delete-menu, delete-plat, activate-plat, add-plat-process, activate-menu, review-management, update-review-status, recalculer-prix-common, error-ban, contact-material-client	Espace et actions opérationnels communs partagés entre le personnel et la direction (gestion des menus, des plats, des commandes et modération des avis).
Administrateur	dashboard-admin, stats-admin, rh-admin, rh-admin-create, rh-admin-delete, rh-admin-toggle, ban-user, unban-user	Espace de direction, statistiques globales, gestion des ressources humaines (RH) et des sanctions/bannissements.

7. Flux fonctionnels clés

7.1 Parcours de commande client (steps 0 à 5)



7.2 Cycle de vie d'une commande

Le tableau ci-dessous détaille le cycle de vie d'une commande passée par un utilisateur, de sa validation initiale jusqu'à sa clôture, ainsi que les règles de transition associées :

Statut actuel	Action / Acteur	Statut suivant / Condition
en_attente	Validation (Staff)	→ acceptee
en_attente	Annulation (Client)	→ annulee
en_attente, acceptee, en_preparation, en_cours_livraison	Annulation (Staff)	→ annulee (avec message obligatoire pour prévenir le client)
acceptee	Lancement (Staff)	→ en_preparation
en_preparation	Départ (Staff / Livreur)	→ en_cours_livraison
en_cours_livraison	Livraison (Staff / Livreur)	→ livree
livree	Contrôle & retour matériel	→ en_attente_retour_materiel (si matériel) → terminee
en_attente_retour_materiel	Réception effective (Staff)	→ terminee

Règles métier associées aux transitions :

- Chaque transition déclenche un appel à OrderManager::updateStatus().
- Une trace est enregistrée via une insertion dans la table vg_commande_statut_historique.
- Une notification par e-mail est envoyée au client.
- Toute annulation entraîne automatiquement la restitution du stock (quantite_restante + nombre_personne).

7.3 Modification, annulation et avis

- **Client** : Toute modification ou annulation est soumise à un contrôle de propriété strict (vérification du filtre `utilisateur_id`), exige un statut initial `en_attente`, et s'appuie sur un recalcul dynamique des prix en AJAX.
- **Staff (Employé / Administrateur)**: mêmes règles sans la contrainte de propriété, avec l'utilisation de transactions SQL sécurisées garantissant un retour en arrière (*rollback*) en cas d'erreur.
- **Avis** : Strictement réservé aux commandes au statut terminée, modéré par le staff avant affichage sur l'accueil.

8. Diagramme de cas d'utilisation

Vue synthétique des acteurs (Visiteur, Utilisateur, Employé, Administrateur) et de leurs interactions avec le système et les services externes (Brevo, OpenRouteService).

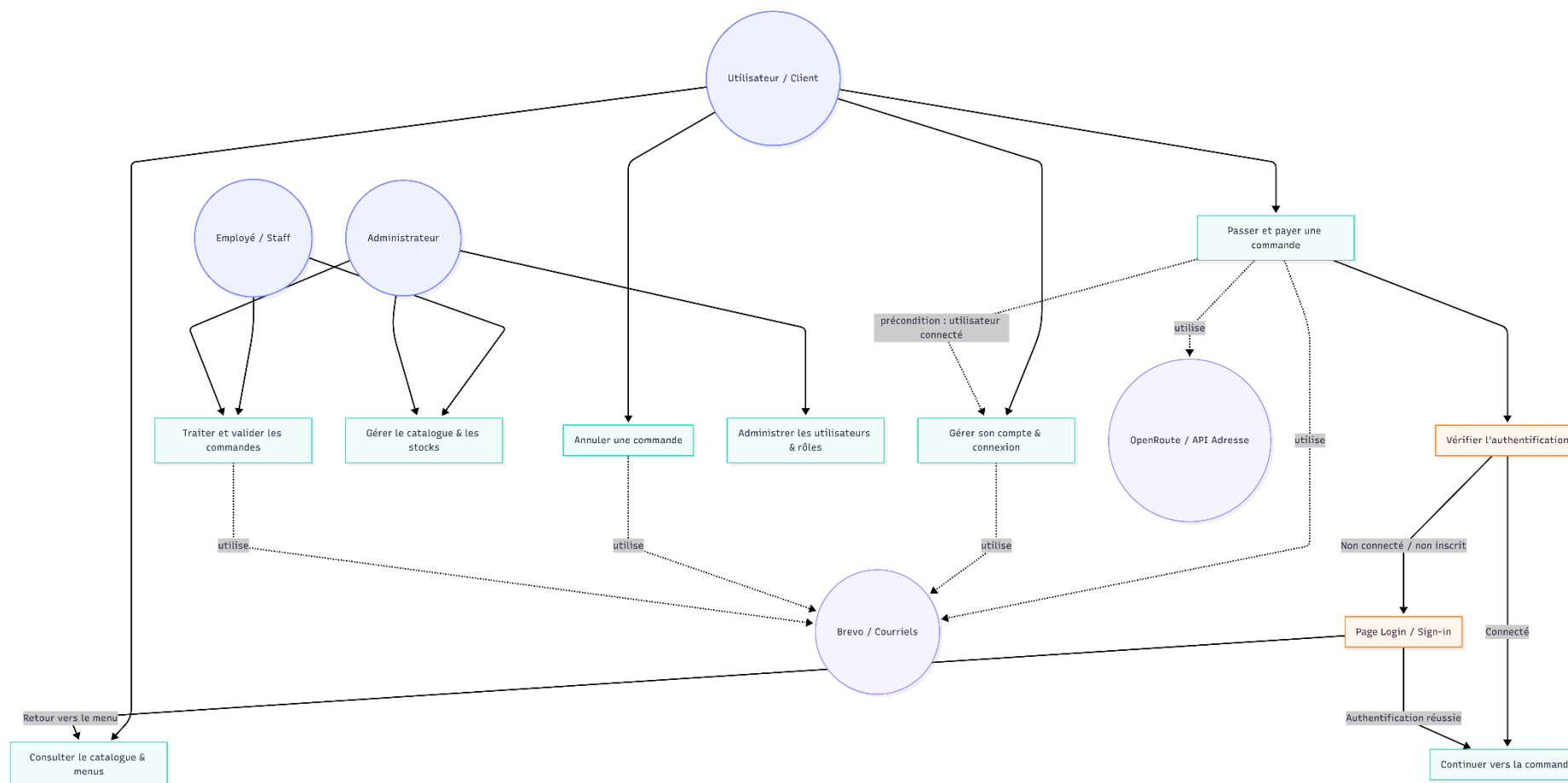


Figure 2 - Diagramme de cas d'utilisation de Vite & Gourmand

9. Diagramme de séquence

Illustration détaillée du parcours de commande, de la saisie de l'adresse jusqu'à la confirmation par email, avec les branches de secours (fallback Haversine, échec d'envoi d'email) .

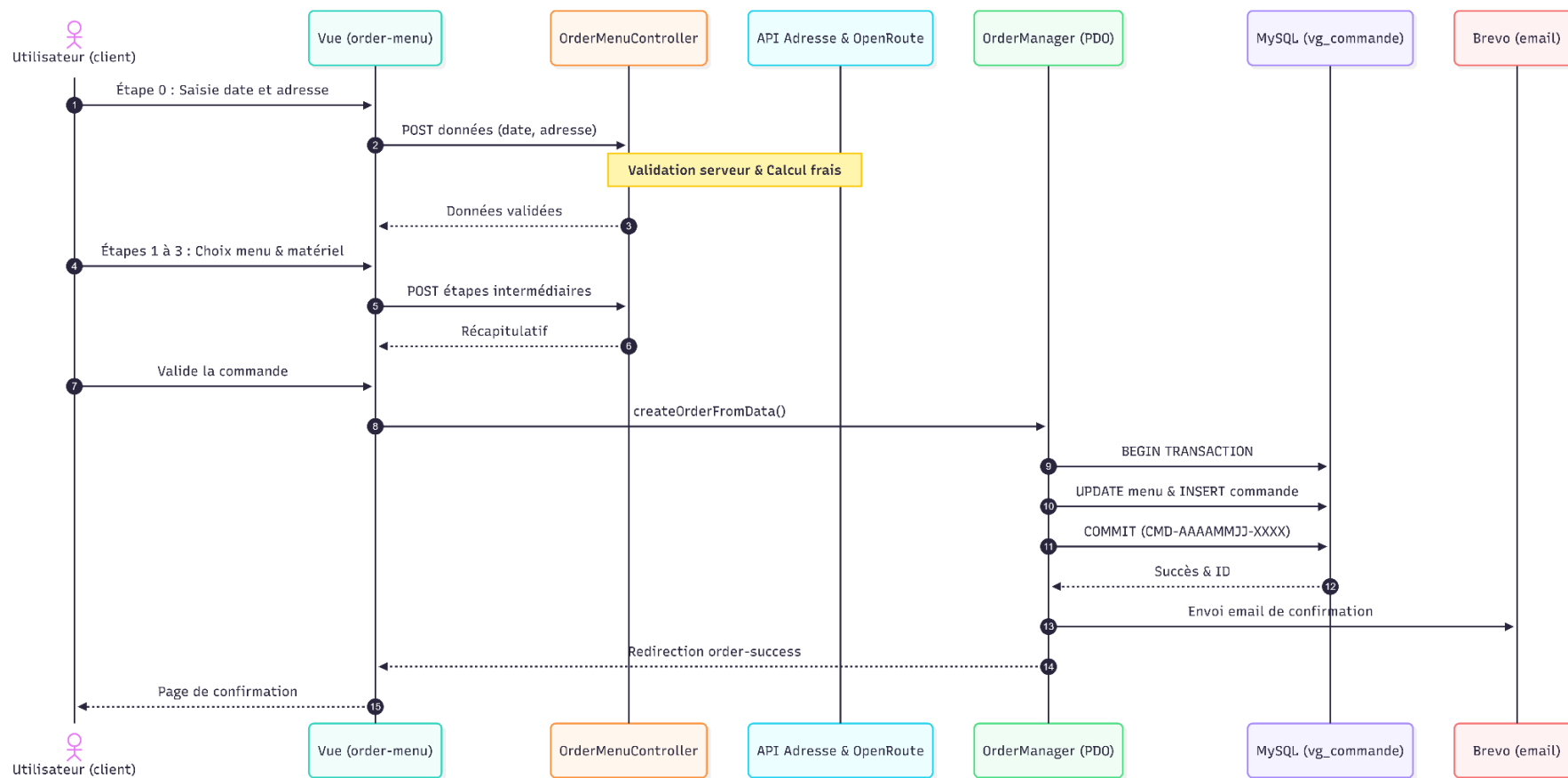


Figure 3 — Diagramme de séquence du parcours de commande / Cas d'erreur gérés côté serveur (non représentés ici) : validation refusée si données invalides · fallback Haversine si OpenRouteService timeout (4s/8s) · rollback transaction si stock insuffisant · échec d'envoi email journalisé sans bloquer la commande.

10. Intégrations externes

L'application s'appuie sur plusieurs services tiers et briques externes, dont la résilience est systématiquement assurée par des mécanismes de repli (*fallback*) ou un fonctionnement non bloquant :

Service	Usage	Résilience
API Adresse (data.gouv.fr)	Autocomplétion et géocodage adresse → coordonnées	Filtre housenumber, fallback manuel
OpenRouteService	Distance routière réelle pour le calcul des frais de livraison	Timeout 4s/8s → repli formule de Haversine
Brevo	Emails transactionnels (commande, reset mdp, création compte staff)	Timeout Guzzle 5s/10s → journalisation <code>error_log</code> , non bloquant
MongoDB	Statistiques analytiques complémentaires (CA par menu/période)	Fonctionnement dégradé mais stable si Mongo indisponible

11. Performances et optimisation

Afin de garantir une expérience utilisateur fluide, un bon référencement et des temps de chargement réduits, plusieurs leviers d'optimisation et règles d'hygiène ont été appliqués :

Domaine	Mesure
Images	Format <i>WebP</i> (~1000px carte, ~1200px hero), compression q85, nom aléatoire
Cache HTTP	<i>public/.htaccess – mod_expires : images (webp/png/jpeg) 1 an, CSS/JS 1 mois.</i> <i>mod_headers : Cache-Control: public, max-age=31536000</i> , immutable sur les images (noms de fichiers aléatoires donc jamais réutilisés), <i>Cache-Control: public, max-age=2592000</i> (sans immutable) sur CSS/JS pour permettre leur mise à jour sans cache-busting.
JS/CSS	Chargement asynchrone, pas de <code><script></code> bloquant
Hygiène de code	Aucun <code>var_dump/print_r/TODO</code>

12. Procédure de déploiement

Le déploiement de l'application en production repose sur une procédure standardisée garantissant la sécurité des configurations, l'intégrité de la base de données et la bonne gestion des environnements :

12.1 Prérequis serveur

- Compte Heroku
- Heroku CLI installée et authentifiée
- Add-on JawsDB MySQL (MySQL non natif sur Heroku)
- Extension PHP *mbstring* déclarée dans `composer.json` (*ext-mbstring*) - absente par défaut du buildpack Heroku sans déclaration explicite.
- Client SQL compatible SSL pour se connecter à JawsDB (HeidiSQL, Dbeaver...) - les identifiants de connexion sont récupérables via **`heroku config:get JAWSDB_URL`**.

12.2 Étapes de mise en production

1. Créer l'app Heroku : **`heroku create vite-gourmand`**
2. Ajouter le buildpack PHP + Procfile (`heroku-php-apache2 public/`)
3. Provisionner JawsDB : **`heroku addons:create jawsdb:kitefin`**
4. Renseigner les Config Vars (équivalent du `.env`) : **`APP_ENV`**, **`MONGODB_URI`**, **`BREVO_API_KEY`**, **`OPENROUTESERVICE_API_KEY`**, **`APP_URL`**... (**`JAWSDB_URL`** est fournie automatiquement par l'add-on)

(Note : quand **`APP_ENV=production`**, `public/index.php` désactive automatiquement **`display_errors/display_startup_errors`** ; les erreurs restent tracées via **`error_log`**.)

Accès réseau MongoDB Atlas : les dynos Heroku standards ne disposant pas d'adresse IP sortante fixe, l'adresse IP utilisée pour se connecter à MongoDB Atlas peut changer à chaque redémarrage du dyno (notamment après chaque déploiement).

Il est donc recommandé d'autoriser l'ensemble des adresses IP (0.0.0.0/0) dans la configuration Network Access d'Atlas, plutôt que de maintenir une liste d'IP spécifiques — la sécurité de la base reposant alors sur les identifiants de connexion (**`MONGODB_URI`**) plutôt que sur le filtrage réseau.

L'absence de cette configuration se manifeste par une erreur de connexion trompeuse (*TLS handshake failed*), pouvant à tort faire penser à un problème de version PHP/OpenSSL plutôt qu'à un filtrage IP — comme rencontré lors de la migration vers PHP 8.4 (voir section 6.2).

5. Importer *Schema_VG.sql* puis *fixture_VG.sql* dans la base JawsDB retirer au préalable les instructions CREATE DATABASE/USE si présente, JawsDB provisionnant sa propre base nommée automatiquement (via le client *mysql* en local, pointé sur l'hôte JawsDB)
6. **git push heroku main**
7. Vérifier les logs : **heroku logs --tail**
8. Tester le parcours complet en production

Docker, présenté en section "*Configuration de l'environnement de travail*", constitue un environnement de développement local.

Bien qu'Heroku propose une option de déploiement par conteneurs Docker (*heroku.yml*, *Container Registry*), ce projet utilise le système de build par défaut d'Heroku (*buildpacks*), conformément à la recommandation officielle qui réserve l'approche conteneurisée aux cas d'usage avancés nécessitant des images personnalisées.

12.3 Politique de sauvegarde à mettre en place

- Base de données : Dump MySQL automatisé quotidien.
- Fichiers médias : Les images de démonstration fournies avec le projet (*storage/uploads/plats/*) sont versionnées dans le dépôt Git. Les nouvelles photos uploadées par le staff en production, dans ce même répertoire, échappent en revanche au contrôle de version (répertoire non synchronisé avec Heroku à chaque déploiement) — un archivage régulier de ce répertoire reste recommandé pour ne pas perdre les photos ajoutées après la mise en production initiale.

13. Maintenance et bonnes pratiques

Afin de garantir la pérennité, la lisibilité et l'évolutivité du code source au-delà du cadre du projet ECF, les règles de maintenance suivantes sont appliquées :

Tâche	Procédure
Ajouter une page	- Ajouter la clé dans <i>app/Config/pages.php</i> (liste existante + rôle concerné) - Le match dans le routeur <i>public/index.php</i> - La méthode dans le contrôleur - La vue dans <i>app/Views/<role>/.</i>
Ajouter un Manager	Créer <i>app/Managers/XxxManager.php</i> (namespace <i>App\Managers</i>), méthodes <i>public static</i> prenant (<i>\PDO \$db</i>) en premier paramètre, requêtes préparées + <i>try/catch</i>. Les Managers gèrent l'accès aux données (requêtes préparées via PDO, transactions) et l'orchestration métier.
Ajouter une méthode métier	La placer dans le Model du domaine concerné (ex. <i>app/Models/Xxx.php</i>), qui porte la logique de calcul pure (tarification, distances) sans accès direct à la base.
Ajouter un thème/régime	Insérer dans <i>vg_theme/vg_regime</i>. Les filtres de recherche (<i>search.Menu.view.php</i>) se génèrent automatiquement via <i>MenuManager::getAllThemes()/getAllRegimes()</i>, aucune modification de vue nécessaire.
Messages utilisateur	error_message(\$msg) / success_message(\$msg) côté PHP, ou afficherFlashJS(\$msg, type) côté JS.
Debug	error_log() en production ; jamais de var_dump ; php -l avant chaque commit.
Accès direct à la base de production	Connexion via HeidiSQL (SSL activé) avec les identifiants JAWSDB_URL
Veille de sécurité des dépendances	Exécuter composer audit avant chaque déploiement pour détecter les vulnérabilités connues (CVE) dans les packages installés. En cas d'alerte, évaluer le risque réel dans le contexte d'usage du projet avant de décider d'une mise à jour (ex. CVE-2026-81525 sur mongodb/mongodb, jugée sans impact car les noms de bases/collections sont fixes, jamais issus d'une saisie utilisateur).
Modifier le Dockerfile	Après toute modification, reconstruire avec docker compose build --no-cache pour éviter qu'une couche mise en cache masque le changement. Toute modification de <i>composer.json</i> nécessite un rebuild (le cache Composer n'est invalidé que si ce fichier change).
Réinitialiser la base Docker	Les données MariaDB sont conservées dans le volume nommé <i>db_data</i>, persistant entre les redémarrages. Pour repartir d'une base vide : docker compose down -v (le -v supprime aussi les volumes), puis réimporter <i>Schema_VG.sql/fixtures_VG.sql</i>.

Tâche	Procédure
Montée de version PHP	Migration effectuée de PHP 8.2 vers 8.4 (<i>voir composer.json, config.platform.php</i>). Vérifier avant toute montée de version l'absence de nouveaux avertissements via heroku logs --tail après déploiement, et tester l'ensemble des parcours (connexion, tunnel de commande, upload, statistiques admin).
Connexion MongoDB Atlas en production	Les dynos Heroku ne disposent pas d'IP sortante fixe, vérifier que Network Access (MongoDB Atlas) autorise 0.0.0.0/0 plutôt qu'une IP spécifique, qui deviendrait invalide au prochain redémarrage du dyno. Une erreur "<i>TLS handshake failed</i>" lors de la connexion est le symptôme de ce blocage, pas un problème de version PHP/OpenSSL.